Improving ML-KEM and ML-DSA on OpenTitan

Efficient Multiplication Vector Instructions for OTBN

Ruben Niederhagen[1,2] and Hoang Nguyen Hien Pham[3,4]

[1] Academia Sinica, Taipei, Taiwan,
ruben@polycephaly.org
[2] University of Southern Denmark, Odense, Denmark
[3] Max Planck Institute for Security and Privacy, Bochum, Germany
[4] Université Grenoble Alpes, Saint-Martin-d'Hères, France,
nguyenhien.phamhoang@gmail.com

Abstract. This work improves upon the instruction set extension proposed in the paper "Towards ML-KEM and ML-DSA on OpenTitan", in short OTBN_{TW}, for OpenTitan's big number coprocessor OTBN. OTBN_{TW} introduces a dedicated vector instruction for prime-field Montgomery multiplication, with a high multi-cycle latency and a relatively low utilization of the underlying integer multiplication unit. The design targets post-quantum cryptographic schemes ML-KEM and ML-DSA, which rely on 12-bit and 23-bit prime field arithmetic, respectively. We improve the efficiency of the Montgomery multiplication by fully exploiting existing integer multiplication resources and move modular multiplication from hardware back to software by providing more powerful and versatile integer-multiplication vector instructions. This enables us not only to reduce the overall computational overhead through lazy reduction in software but also to improve performance in other functions beyond finite-field arithmetic. We provide two variants of our instruction set extension, each offering different trade-offs between resource usage and performance. For ML-KEM and ML-DSA, we achieve a speedup of up to 17% in cycle count, with an ASIC area increase of up to 6% and an FPGA resource usage increase of up to 4% more LUT, 20% more CARRY4, 1% more FF, and the same number of DSP compared to OTBN_{TW}. Overall, we significantly reduce the ASIC time-area product, if the designs are clocked at their individual maximum frequency, and at least match that of OTBN_{TW}, if the designs are clocked at the same frequency.

Keywords: OpenTitan · PQC · ML-KEM · ML-DSA · ISE · HW/SW co-design

1 Introduction

NIST's selection of the lattice-based schemes ML-KEM and ML-DSA as post-quantum cryptography (PQC) standards in August 2024 marked a pivotal milestone in the global transition to post-quantum cryptography. To prepare embedded systems for this shift, these algorithms have been extensively studied across a wide range of implementation platforms. These include pure software implementations targeting architectures such as Cortex-M4 [HZZ+22, HAZ+24, BRS22, AHKS22, GKS21, ABCG20, BKS19], Arm Neon [BHK+22], and RISC-V [ZYHK25]; hardware/software co-designs [LTQ+24, LQYW24, ZXXH22]; and pure hardware implementations [NDD+25, BNG21]. Among these, hardware/software co-design approaches leveraging instruction set extensions (ISEs) have gained popularity. By accelerating computationally intensive operations with dedicated hardware support, they typically offer significant improvements in execution time with only a modest increase in hardware resource requirements.

OpenTitan, an open-source hardware root of trust (RoT), has gained community attention recently for its claim to support secure classical cryptographic acceleration, particularly RSA [RSA78] and ECC [Mil86, Kob87], on its OpenTitan Big Number (OTBN) coprocessor, whose architecture is heavily inspired by that of RISC-V and whose implementation is hardened with multiple countermeasures against side-channel attacks. This opens the research question to extend OTBN for PQC support, which was explored in [AOP+25], where the authors proposed a vectorized ISE for OTBN. Their extensions, while generic, proved effective in significantly accelerating both ML-KEM and ML-DSA compared to using only the original OTBN's big number instructions. Although their work demonstrated impressive results and engineering effort, there remains space for improvement for some of their hardware design choices.

A particularly interesting decision in their work was to extend the existing 64-bit multiplier of OTBN to a hardware multiplier performing modular Montgomery multiplication on vectors of 16-bit or 32-bit words. This certainly makes sense, given the fact that OTBN has a wide special register `MOD` for specifying the modulus for some modular computations, specifically, for 256-bit modular addition `bn.addm` and subtraction `bn.subm`, but not for large-integer multiplication, which is implemented in software. However, the work in [AOP+25] did not investigate an alternative, more traditional approach, i.e., performing modular vector multiplication also in software.

In this paper, we explore this design alternative and show that it is more efficient in terms of time-area product with moderate increase in hardware cost and improved software performance compared to the design in [AOP+25]. In addition, our proposed ISE for OTBN is more versatile and hence well suited for lattice-based cryptography in particular and PQC in general.

Related Work. Over the path of the NIST standardization and since the selection of the first NIST PQC standards, there has been a vast amount of ML-KEM and ML-DSA implementations. We limit the scope of the discussion for this paper to embedded devices. We compare our results to the baseline design [AOP+25] as well as to [SOSK23, US25, Tur23] on OpenTitan. To provide a more general context, we also compare our results to implementations on Cortex-M4 [HZZ+22, HAZ+24] and RISC-V [NDMZ+21, FSS20, KSFS24, LQYW24, LTQ+24, ZYHK25].

Open Source. We would like to note that this research is made possible thanks to OpenTitan and [AOP+25] being open-source[1] under permissive licenses. All hardware and software described in this paper can be found at `https://github.com/PQC-OpenTitan/improving-ml-kem-and-ml-dsa-on-opentitan` under Apache 2.0 license.

Contributions. We propose three main improvements to the ISE in [AOP+25]:

1. We propose a new multiplication vector instruction that is more generic and has a higher throughput than that in [AOP+25]. By shifting our design rationale to bring modular multiplication back to the software side, instead of a hard-coded Montgomery multiplier as in [AOP+25], in our first design variant, we reduce the number of cycles per non-modular vector multiplication from four to two, and per Montgomery vector multiplication from twelve to seven for both 32- and 16-bit vector inputs [AOP+25, Section 6.2.2].

2. The vectorized multiplier of [AOP+25] does not make full use of existing resources of OTBN: it computes only four 16-bit multiplications per cycle. We further extend this to support sixteen 16-bit multiplications per cycle in a second variant of our new multiplication instruction, dropping the number of cycles per Montgomery multiplication to only four for the 16-bit case.

[1] `https://github.com/PQC-OpenTitan/towards-ml-kem-and-ml-dsa-on-opentitan`

3. The multi-word vectorized adders of [AOP+25] for addition/subtraction vector instructions ripple-carry bits over word boundaries to construct larger from smaller adders. This has a negative effect on the critical path. We discuss and evaluate several approaches to implement more efficient hardware vector adders.

Paper Structure. This paper is organized as follows: Section 2 provides background information on ML-KEM, ML-DSA, NTT, and OpenTitan, a summary of [AOP+25], and our terminology. Section 3 introduces the performance metrics and the evaluation methodology used for our results, which serve as the basic for comparison with [AOP+25]. Section 4 explains in detail how the adders and multiplier of OTBN work and how they are extended for [AOP+25]. We also describe the two variants of our new multiplication instruction and their hardware performance on both ASIC and FPGA. Section 5 presents the software differences between the two multiplication variants for Montgomery multiplication, focusing on implementation details such as lazy reduction and cycle counts. We investigate how the generic design can be used in other parts of the schemes as well and discuss its performance impact. Section 6 concludes our work with the overall cost of the design for both hardware and software.

2 Preliminaries

We follow the notation NIST FIPS 203 [Nat24a] for ML-KEM and 204 [Nat24b] for ML-DSA. In addition, we define $[r]_d$ to be the unique element r' in the range $[0, 2^d)$ such that $r' \equiv r \bmod 2^d$ and denote $\lfloor \frac{r}{d} \rfloor$ as $[r]^d$, with $d \in \mathbb{N}$. For the rest of the paper, $n = 256$, $d = 16$ for ML-KEM and $d = 32$ for ML-DSA, if not stated otherwise.

2.1 NIST PQC Standards

ML-DSA. The quantum-resistant digital signature scheme Dilithium [DKL+18, LDK+22] is standardized under the name Module-Lattice-based Digital Signature Algorithm (ML-DSA) as specified in FIPS 204 [Nat24b]. It is constructed following the Fiat-Shamir transformation with aborts [Lyu09]. Its security relies on the hardness of the standard Module Learning with Errors (MLWE) and Module Short Integer Solution (MSIS) lattice problems. Arithmetic in ML-DSA is over the polynomial ring $R_q = \mathbb{F}_q[X]/\langle X^n + 1\rangle$ with $n = 256$ and the 23-bit prime $q = 8380417$. It covers three security levels ML-DSA-44, ML-DSA-65 and ML-DSA-87. For more details, we refer to [Nat24b].

ML-KEM. The quantum-resistant key encapsulation mechanism Kyber [SAB+22] is standardized under the name Module-Lattice-based Key-Encapsulation Mechanism (ML-KEM) as specified in FIPS 203 [Nat24a]. Its security is based on the MLWE problem scaled for different parameter sets through the rank k of the module. The polynomial ring used in ML-KEM is also of the form $R_q = \mathbb{F}_q/\langle X^n + 1\rangle$ with $n = 256$ as in ML-DSA but with the 12-bit prime $q = 3329$. It also offers three security levels ML-KEM-512, ML-KEM-768 and ML-KEM-1024. For more details, we refer to [Nat24a].

2.2 Number Theoretic Transform

Polynomial multiplication can be efficiently implemented using Cooley–Tukey (CT) [CT65] and Gentleman–Sande (GS) [GS66] algorithms—also known as fast Fourier transform (FFT) algorithms—in $\mathcal{O}(n \log n)$, instead of $\mathcal{O}(n^2)$ when using the general schoolbook method for $R_q = \mathbb{F}_q[X]/\langle X^n + 1\rangle$. The number-theoretic transform (NTT) is the discrete Fourier transform (DFT) on finite fields.

Assume that a primitive $2n$-th root of unity ζ exists. Then we have the ring isomorphism $R_q \cong \prod_{i=0}^{n-1} \mathbb{F}_q[X]/\langle X - \zeta^{2i+1}\rangle$. The forward and backward mapping are denoted as NTT

Algorithm 2.1: Montgomery multiplication [Mon85].

Input : $a, b \in [0, q)$, $q \in \left(0, 2^d\right)$, $R = \left[-q^{-1}\right]_d$
Output : $r = ab(2^{-d}) \mod q$ and $r \in [0, q)$

$c = ab$
$t = [[c]_d R]_d$
$r = [c + tq]^d$
if $r \geq q$ **then return** $r - q$
return r

and INTT respectively, where the latter stands for inverse number-theoretic transform. The roots of unity are called "twiddle factors". Multiplication of two polynomials $a, b \in R_q$ is "pointwise" multiplication of their "NTT representations":

$$a \cdot b = \mathsf{INTT}(\hat{a} \circ \hat{b}) = \mathsf{INTT}(\mathsf{NTT}(a) \circ \mathsf{NTT}(b)).$$

Polynomial multiplications in ML-DSA and ML-KEM use this approach. However, while ML-DSA has $\log n = 8$ NTT layers, corresponding to a full split of the ring, ML-KEM has only seven layers due to the absence of a $2n$-th root of unity. This implies that the multiplication inside NTT domain is not pointwise, but on 128 linear polynomials—thus also referred to as "pair-pointwise", as follows, for $i \in [\![0, \frac{n}{2} - 1]\!]$,

$$(\hat{a}_{2i} + \hat{a}_{2i+1}X)\,(\hat{b}_{2i} + \hat{b}_{2i+1}X) = (\hat{a}_{2i}\hat{b}_{2i} + \hat{a}_{2i+1}\hat{b}_{2i+1}\zeta^{2\mathsf{br}_7(i)+1}) + (\hat{a}_{2i}\hat{b}_{2i+1} + \hat{a}_{2i+1}\hat{b}_{2i})X.$$

2.3 Modular Multiplications

As shown in Section 2.2, all polynomial arithmetic ultimately relies on modular arithmetic, notably multiplication, as the lowest-level building block. Algorithm 2.1 shows Montgomery modular multiplication [Mon85]. It was chosen as the hardware multiplier in [AOP+25] since the two operands are of the same size, making it suitable for vectorized multiplication instruction, unlike other algorithms, such as Plantard [Pla21] or its improved version [HZZ+22], where the size of one operand is double that of the other one in exchange for one multiplication less. We also use Montgomery multiplication in this work since we aim for generic multiplication on same-width inputs. We deliberately exclude Barrett multiplication [Bar86] because it requires bit shifts that do not align with word sizes, introducing additional cost. In contrast, Montgomery multiplication uses word-size truncation, making it more suitable for generic multiplication designs.

2.4 OpenTitan

OpenTitan is a RISC-V-based open-source RoT [Ope23], which consists of a main 32-bit Ibex RISC-V core and a big number coprocessor, called OTBN, accelerating classical asymmetric cryptography, including RSA [RSA78] and ECC [Mil86, Kob87]. It offers various cryptographic modules, such as Keccak message authentication code (KMAC), HMAC-SHA2, AES [AES01] and cryptographically secure random number generator (CSRNG) with an entropy-source module enabling the generation of deterministic or true random numbers compliant to, e.g., NIST standards.

OTBN. OTBN [Ope23] is hardened against multiple side-channel attacks [Ope23, Section 7.2.2, Section 8.2.2]. Its countermeasures include data-path blanking for preventing power leakage over unused data path, Hsiao integrity-protection code [Che08] for protecting data from being tampered, and secure wiping of its internal states and memories [Ope24] for clearing leftover leakage.

OTBN has two instruction sets: a 32-bit RISC-V-based "base instruction subset" with 32 general-purpose registers (GPRs) for application control flow, and a "big number (bn) instruction subset" with 32 wide data registers (WDRs) `w0` to `w31` of 256-bit width. Additionally, there are control and status registers (CSRs) and wide special-purpose registers (WSRs) that give access to randomness sources, arithmetic flags, key material, a special "modulus register" `MOD`, and an "accumulator register" `ACC` [Ope23, Section 8.2].

There are two main building blocks in OTBN, big number arithmetic logic unit (BN-ALU) and big number multiply accumulate unit (BN-MAC). The latter is responsible for unsigned multiplication instructions (`bn.mulqacc` and its variants), while the former handles all other big number instructions. The key components involved are the adders and the multiplier of OTBN. BN-MAC contains a 64-bit multiplier and a 256-bit adder for accumulation of the multiplication result to the `ACC` register. BN-ALU includes two 256-bit adders, called X and Y, for modular addition (`bn.addm`) and subtraction (`bn.subm`, implemented by adding the two's complement of the subtrahend). In the following, Q is a modulus defined in `MOD` and A, B are source operands. For `bn.addm`, adder X computes $X = A + B$ and adder Y computes $Y = X - Q = X + \sim Q + 1$ ($\sim$ is bit negation); for `bn.subm`, adder X computes $X = A - B$ and adder Y computes $Y = X + Q$. The final result is X or Y depending on their most significant bit (MSB).

KMAC Block. OpenTitan's KMAC core supports Keccak-based message authentication codes (MACs), unauthenticated SHA-3, and (c)SHAKE [Ope23, Section 9.13], with a synthesis-time option for enabling or disabling first-order masking [Ope23, Section 9.13.1] using Domain-Oriented Masking [GSM17] and masking data before loading them into Keccak. KMAC is considered masked throughout this paper as in [AOP+25].

2.5 Previous Variant

[AOP+25] proposes the following extensions to OTBN:

- Vectorized shift instruction: `bn.shv{.16H,.8S}`.
- Vector transpose instruction: `bn{.trn1,.trn2}{.16H,.8S}`.
- Vectorized addition/subtraction: `bn.{addv,subv}(m){.16H,.8S}`.
- Vectorized multiplication: `bn.mulv(m)(.l){.16H,.8S}`.
- Direct connection from OTBN to the KMAC core, called the KMAC interface.

The first four instructions are integrated into the BN-ALU module of OTBN since it already has shifting units and adder resources. For vectorized addition and subtraction, the two 256-bit adders X and Y (cf. Section 2.4) are extended to support either one 256-bit, eight single-word 32-bit (`.8S`) or sixteen half-word 16-bit (`.16H`) additions/subtractions, called "configurable vectorized adder" [AOP+25, Section 6.1.1]. These instructions follow the same logic of `bn.addm`/`bn.subm` of OTBN as explained in Section 2.4, but individually for every vector element. The modulus is stored in the `MOD` register. The four instructions above all take one cycle each. The vectorized multiplication is integrated into the BN-MAC unit according to the main approach of [AOP+25], where they aimed to reuse most of hardware resource of OTBN. They extend the existing 64-bit multiplier of BN-MAC to perform either one 64-bit, two 32-bit, or four 16-bit multiplications, called "configurable vectorized multiplier" [AOP+25, Section 6.1.2]. It requires four cycles for a complete 32-/16-bit unsigned integer vector multiplication [AOP+25, Section 6.2.2]. They hold execution of the fetch unit and use a state machine to perform modular Montgomery multiplication (cf. Algorithm 2.1) with a total instruction latency of twelve cycles. The modulus q and the constant R are again stored in the `MOD` register.

2.6 Terminology

For clarity, we distinguish between five hardware designs with corresponding software versions throughout this paper:

- **$\text{OTBN}_{\text{base}}$**: Unmodified OpenTitan OTBN hardware design.
- **OTBN_{TW}**: The whole hardware design of [AOP+25], including the KMAC interface and the ISE mentioned in Section 2.5.
- **$\text{OTBN}_{\text{KMAC}}$**: $\text{OTBN}_{\text{base}}$ with only the KMAC interface of OTBN_{TW}.
- **OTBNV1/OTBNV2**: Hardware design for the first (OTBNV1) and second variant (OTBNV2) of our new multiplication instruction, as described in Section 4, including the KMAC interface and the BN-ALU instructions of OTBN_{TW}.

3 Methodology and Metrics

This section describes the hardware and software performance metrics used for comparisons with other works, and provides guidance on interpreting the hardware-related results.

3.1 Performance Metrics

For software, we only compare cycle counts and code size, as these are the most relevant metrics for our design changes over OTBN_{TW}. Stack usage remains unchanged from [AOP+25], so it is not reported. For hardware, we differentiate metrics between FPGA and ASIC:

- FPGA: We collect resource consumption in terms of Lookup Tables (LUTs), Digital Signal Processors (DSPs), adder resources (CARRY4), and Flip Flops (FFs) as well as the maximum frequency (F_{max}) following the processes described below.
- ASIC: We collect area resources and F_{max} frequency of the designs. For the synthesis tool Cadence Genus, area is defined as the "Total Area" field in the synthesis report. For the synthesis tool OpenRoad, we report "design_area". In both cases, RAM is treated as a black box and not included in the area estimates.

Maximum Frequency. We obtain the maximum frequency of the designs using a binary search: We start at a low start frequency and then double the frequency until timing fails. Then we iteratively update the midpoint between the highest successful and the lowest unsuccessful frequency. We terminate the process once lower and higher frequencies are only 2.5% apart and return the highest successful frequency as maximum frequency F_{max}.

Time-Area Product. We also discuss the time-area product of our hardware/software co-design for two cases: First, we define time as the cycle count of the software divided by F_{max} of the corresponding hardware design; second, we define time just as the cycle count, akin to scenarios where the designs run at the same frequency and not at their individual F_{max}, as it is common in practice. In both cases, area is the area of ASIC synthesis. The time-area product is normalized against OTBN_{TW} as reference.

Metrics between synthesis processes are not equivalent and only provide relative information for design variants synthesized with the same process.

3.2 Evaluation Methodology

For FPGA, we target the ChipWhisperer CW310-K410T (xc7k160tfbg676-1) from the Kintex-7 family—the official OpenTitan board also used in [AOP+25]. Synthesis is done with Xilinx Vivado, and all reported numbers are post-synthesis and post-implementation,

meaning that both floor planning and placement and routing (P&R) have been performed. Therefore, they correspond closely to the actual deployment of the designs on an FPGA.

For ASIC, we provide numbers for two synthesis flows: the commercial tool Cadence Genus with the ASAP7 PDK, as used in [AOP+25], and the open-source tool OpenRoad[2] [ACF+19] with the Sky130HD PDK to simplify reproducibility of our results without requiring a Cadence Genus license. Using two different flows increases the reliability of our results if the relative performance trend is consistent across tools and technologies. Results of Cadence Genus are post-synthesis without P&R, which is a common choice and also used in [AOP+25]. For OpenRoad, we report post clock-tree synthesis results, which is past placement but does not include final routing. Doing sign-off P&R for ASIC design is very time-consuming and is out of the scope of this work. This means that our ASIC results for resource consumption and maximum frequency serve only as a lower bound estimates. We argue that relative performance within the same synthesis flow is sufficient for comparing different design choices.

We synthesized all the hardware variants in Section 3.1 and benchmarked the corresponding software versions using this evaluation methodology to obtain comparable results in equivalent metrics within one synthesis flow. Due to small differences in the codebase and the synthesis process, our reported numbers slightly differ from those in [AOP+25], but only within a small margin.

4 Hardware Design

This section describes the low-level designs for vector arithmetic (addition and multiplication) as well as the design rationale and the hardware implementation of our ISE.

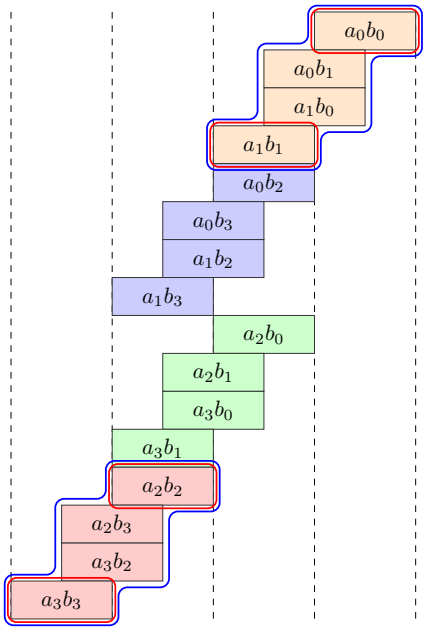
4.1 Vector Arithmetic

Multiplier. The 64-bit hardware multiplier, located in BN-MAC, is a performance-critical component of OTBN (in principle for all discussed variants) since it contributes the largest share of the logic resource cost and since it is in the critical path of the overall design. The SystemVerilog implementation of the $\text{OTBN}_{\text{base}}$ design simply uses the multiplication operator `res = a * b` for the unsigned multiplier and leaves it to the synthesis tool to implement multiplication efficiently.

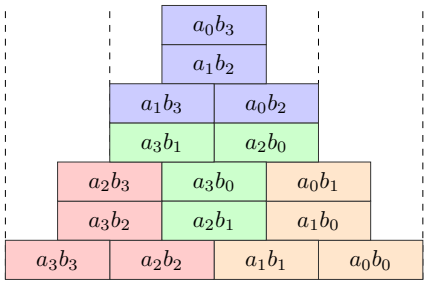
The OTBN_{TW} design in [AOP+25] introduces the vector multiplication instruction `bn.mulv` (with optional reduction as `bn.mulvm`) for 256-bit vectors of sixteen 16-bit or eight 32-bit unsigned integer words using a multi-cycle state machine. To reduce resource cost, their `bn.mulv` instruction shares multiplier resources with the existing 64-bit integer multiplier by decomposing it into sixteen 16-bit integer multipliers. 32-bit and 64-bit multiplication results are then obtained by decomposing 32-bit and 64-bit input operands into 16-bit words and summing up the partial products using a Wallace tree [Wal64].

Figure 1a shows the OTBN_{TW} approach: The complete 128-bit result is obtained by summing up all partial 32-bit products. Two 32-bit multiplication results are obtained with the same adder logic from the first and last four partial product groups (marked in blue) by setting the "middle" partial products a_0b_2, $a_0b_3, \ldots a_3b_1$ to zero. Four 16-bit multiplication results are obtained by setting all partial products except a_0b_0, a_1b_1, a_2b_2, and a_3b_3 (marked in red) to zero. The 16-bit and 32-bit multiplication results cannot have a carry; hence, the same Wallace tree and final 128-bit adder logic is used for the 16-, 32-, and 64-bit operands. These design choices in the OTBN_{TW} design have the following resource and performance implications:

[2] https://theopenroadproject.org/



(a) Multiplier structure of OTBN_{TW}.



(b) Our multiplier structure.

Figure 1: Multiplier structure of OTBN_{TW} in [AOP+25] compared to our multiplier.

- The core multiplier only requires slightly more resources than the $\text{OTBN}_{\text{base}}$ 64-bit multiplier, primarily in the form of some multiplexers to disable partial products for 16- and 32-bit multiplications.
- However, only half of the resources of the $\text{OTBN}_{\text{base}}$ 64-bit multiplier are used for 32-bit multiplications and only a quarter for the 16-bit multiplications, resulting in a relatively low throughput for these operations: Four cycles are required to compute all 16- or 32-bit multiplication results [AOP+25, Section 3.1.1, 6.2.2].

We improve the throughput of the multiplier in our design by exploiting all multiplier resources for the 16- and 32-bit vector multiplications. This requires breaking the $\text{OTBN}_{\text{base}}$ 64-bit multiplier further apart, with a more complex multiplexer structure to select more operands from the input vectors such that all sixteen 16-bit multiplications are computed: For the 16-bit case, we select all sixteen 16-bit input pairs as inputs for the 16-bit multipliers. For the 32-bit case, we either break down the four odd or even 32-bit words into 4×4 16-bit components and add up the partial results accordingly. For the 64-bit case, we break down the 64-bit input pairs into four 32-bit multiplications which then use the 32-bit multiplication logic above and finally add up the partial results.

A resource and performance comparison is shown in Table 1. Our approach requires more logic for organizing the inputs to the sixteen 16-bit multipliers and for composing the output vector than the $\text{OTBN}_{\text{base}}$ design. Also, we need significantly more CARRY4 adder resources to sum up the partial results than $\text{OTBN}_{\text{base}}$ and OTBN_{TW}. Since the sums for computing 32-bit and 64-bit multiplication results are sequential, we achieve a lower F_{max} than $\text{OTBN}_{\text{base}}$ and OTBN_{TW} on the ASIC platforms. To compensate for this loss, we also investigated a variant where the 64-bit multiplication result is computed directly from the 32-bit partial results of the 16-bit multiplications using a Wallace tree. We reduce the depth of the Wallace tree by aligning the 32-bit partial results as shown in Figure 1b. Our Wallace tree requires seven layers, two of which can be reduced in parallel, while the OTBN_{TW} design reduces its sixteen layers purely sequentially. The FPGA resource cost of our Wallace-tree variant is quite significant, while maximum frequency improvements are

Table 1: Resources and F_{max} for the multipliers (F_{max} in MHz, area in μm^2 $\times 10^3$).

Platform	Throughput			FPGA Vivado				ASIC Genus		ASIC ORFS	
	16	32	64	LUT	DSP	CARRY4	F_{max}	Area	F_{max}	Area	F_{max}
OTBN	—	—	1	475	16	24	75	41.0	1,350	159.3	100
$OTBN_{TW}$	4	2	1	1,300	16	26	81	51.9	930	177.4	69
our	16	4	1	1,495	16	72	74	62.4	900	218.2	60
our (Wallace)	16	4	1	2,127	16	76	79	78.1	990	227.0	73

rather moderate (in particular on the FPGA device). To avoid a bias in the performance comparison with the unoptimized $OTBN_{base}$ variant, we will not use our Wallace variant in the remainder of this paper—although the Wallace variant has a similar architecture as the $OTBN_{TW}$ approach and it might be a worthwhile optimization for ASIC implementations if high performance is required.

Adder. Adders have a less significant impact on the overall $OTBN_{base}$ performance compared to the multiplier. There are in total three of them in the $OTBN_{base}$ design for large-integer arithmetic as described in Section 2.4. The $OTBN_{base}$ design simply implements 256-bit addition using the addition operator `res = a + b` and leaves it to the synthesis tools to implement efficient adder logic. For example, Vivado implements the 256-bit adder as Carry-Select Adder (CSLA) using three 128-bit adders, each constructed from 33 CARRY4 primitives.

The $OTBN_{TW}$ design extends the $OTBN_{base}$ design such that the adder resources are shared: 1×256-bit, 4×64-bit, and 8×32-bit additions for the BN-MAC module and 1×256-bit, 8×32-bit, and 16×16-bit additions for the BN-ALU module. However, the design is fairly inefficient in terms of clock latency: $OTBN_{TW}$ simply chains the carry out of 16-bit result i to the carry-in port of addition $i + 1$ depending on the adder mode (i.e., the vector-element word size). Hence, all sixteen additions are effectively serialized and the synthesis tools cannot use any adder architecture that is optimized for large-integer addition. Since BN-ALU chains two 256-bit adders in sequence (cf. Section 2.4), the longest path for Vivado and Genus therefore is in BN-ALU.

We implement the vector adder for our design using a more efficient approach: Our generic vector adder uses a standard approach described, e.g., in [DT05]. At each word boundary, we insert one additional propagate/kill bit in both input operands. If both inserted buffer bits are zero, the carry from the lower vector word is caught and cannot propagate to the next vector word. If one of the buffer bits is one, a carry from the lower vector word propagates to the next vector word and both vector words are effectively combined to a larger vector word. In case a vector element has a carry in (e.g., for implementing subtraction using two's complement), both buffer bits are set to one. Therefore, we need to add fifteen bits to the adders to enable 16-bit, 32-bit, and 64-bit word vector addition. Otherwise, we leave it to the synthesis tool to implement our 271-bit adder.

Another standard approach also described, e.g., in [DT05] is to kill carries between vector words explicitly. This can be efficient, e.g., for prefix-tree architectures with fewer wiring tracks [Har03] such as the Brent-Kung adder [BK82]. The carry-kill approach can also be used when implementing FPGA adders explicitly using CARRY4 primitives. We implemented both approaches to investigate their efficiency.

The comparison in Table 2 shows that the $OTBN_{TW}$ adders have a particularly high clock latency due to their sequential design. The CARRY4 design uses the same 128-bit CSLA adder as the $OTBN_{base}$ design (Vivado default 256-bit adder) but uses additional logic to conditionally kill carries between the vector words. It has a similar resource usage and clock latency as the $OTBN_{base}$ adder. We also show ASIC synthesis results for a 256-bit Brent-Kung adder and a Brent-Kung vector variant that kills carries between the vector words. However, our primary goal is to improve the ISE, rather than to perform

Table 2: Resources and F_{max} for the adders (F_{max} in MHz, area in μm^2 $\times 10^3$).

Platform	Word Sizes	FPGA Vivado				ASIC Genus		ASIC ORFS	
		LUT	DSP	CARRY4	F_{max}	Area	F_{max}	Area	F_{max}
OTBN	256	449	0	99	200	7.2	4,000	28.0	250
$OTBN_{TW}$ ALU	16/32/256	272	0	80	57	4.6	500	21.4	190
$OTBN_{TW}$ MAC	32/64/256	263	0	72	70	4.9	890	19.8	190
our buffer bit	16/32/64/256	476	0	103	210	7.5	3,800	32.3	260
our CSA CARRY4	16/32/64/256	474	0	96	200	—	—	—	—
our Brent Kung	256	682	0	—	175	4.0	2,600	16.6	270
our Brent Kung vec	16/32/64/256	633	0	—	155	4.1	2,200	18.2	260

low-level hardware optimization. Including low-level optimization would both bias the ASIC evaluation of the ISE in our favor and require different designs tailored to each toolchain. Hence, for a fair comparison, we decided to use the same design across all tools in the remainder of this paper, i.e., the buffer-bit approach.

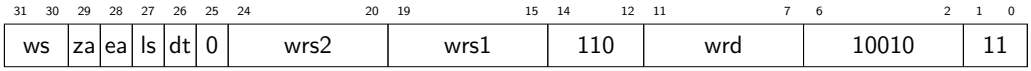
4.2 ISE Design Rationale

OTBNV1: Basic Vector Multiply and Accumulate. Our goal is to define a new vector multiplication instruction that efficiently reuses existing multiplier resources while remaining generic enough for general-purpose multiplication. At the same time, it should provide dedicated features for efficient Montgomery reduction in ML-{KEM,DSA} and more generally, for primes smaller than 16-bit or 32-bit. Therefore, our `bn.mulv` instruction has data-type flags that indicate whether the operation is performed on 32-bit words (`.8S`) or 16-bit words (`.16H`), as in [AOP+25]. Since Montgomery multiplication performs an addition, we can make use of the accumulator in BN-MAC with an `.acc` flag to enable accumulation and a `.z` flag to clear the accumulator before use.

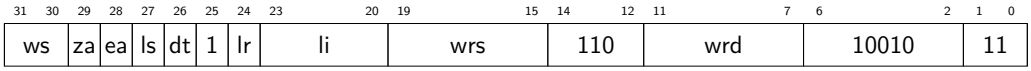
As shown in Section 4.1, the maximum throughput for 32-bit multiplications that we can achieve is four. All four 64-bit outputs then can be stored in the 256-bit accumulator register `ACC` of BN-MAC. Furthermore, although we can compute sixteen 16-bit multiplications, the 256-bit `ACC` register can only accumulate eight 32-bit results, hence restricting the throughput of `.16H` mode to eight. As a result, we use a lane-selector flag in our `bn.mulv` instruction to indicate if the multiplication operates on odd or even vector word lanes.

Montgomery multiplication has one integer multiplication where only the low half of the result is required (Algorithm 2.1, Line 2) and one where only the high half of the result is required (Algorithm 2.1, Line 3). Therefore, we provide a 2-bit output-word selector flag that indicates if the complete double-word result (default, no flag), only the low half (`.lo`), or only the high half (`.hi`) is returned. Since `bn.mulv` operates on even or odd lanes, we simply return the double-width results on the default case. If only one half is returned (`.lo` or `.hi`), we copy the words in the other lanes (even lanes for the odd flag, odd lanes for the even flag) of the first source register without modification into the corresponding output lanes. However, for all output-word selects, all double-word results are accumulated in the `ACC` register if the `.acc` flag is set. This allows us to compute the entire instruction sequence required for Montgomery multiplication first on the even lanes and then on the odd lanes. Figure 2a shows the resulting encoding for our `bn.mulv` instruction.

The NTT and INTT require scalar-vector multiplication. To avoid broadcasting a scalar value to all words of the corresponding vector register, we provide a variant `bn.mulv.l` of our multiplication instruction (as in [AOP+25]) that uses a 4-bit lane index and a dedicated lane register `sw0` or `sw1` (mapped to registers `w16` and `w17`) instead of the second source operand. The encoding of `bn.mulv.l` is shown in Figure 2b.



(a) Encoding for `bn.mulv` with the mnemonic:
`bn.mulv{.16h|.8s}{.even|.odd}[.acc][.z][.lo|.hi] <wrd>, <wrs1>, <wrs2>`



(b) Encoding for `bn.mulv.l` with the mnemonic:
`bn.mulv.l{.16h|.8s}{.even|.odd}[.acc][.z][.lo|.hi] <wrd>, <wrs>, <lreg>.<lidx>`

Figure 2: Encoding for `bn.mulv` and `bn.mulv.l` with data type (dt = 0 for `.16H`, dt = 1 for `.8S`), lane select (ls = 0 for `.even`, ls = 1 for `.odd`), enable accumulator (ea = 1 for `.acc`), zero accumulator (za = 1 for `.z`), word select (ws = 00_2 for double-length result, ws = 01_2 for `.lo`, ws = 10_2 for `.hi`), lane index `<lidx>` (li), and lane register `<lreg>` (lr = 0 for `sw0` (`w16`), lr = 1 for `sw1` (`w17`)). For version 2, there is no lane select (`.even`|`.odd`) for `.16H` when `.hi` or `.lo` is active (as always all lanes are being processed), but it is required otherwise.

OTBNV2: Second Accumulator Register. As described above, the main limiting factor that prevents us from using all of the multiplier resources for 16-bit word vector multiplications is the `ACC` register. Due to the output-word selector flag, it actually is possible to return all low or high parts of the 16-bit multiplications in a single cycle. However, in Montgomery reduction, we have to keep the double-length results of the initial multiplications (cf. Algorithm 2.1, Line 1) so that they can be added to the results of the third multiplication (cf. Algorithm 2.1, Line 3).

In order to exploit all multiplier resources for the 16-bit word type, we propose to double the `ACC` register width. By introducing a second accumulator register `ACCH`, we obtain a total accumulator register width of 512-bit, which allows us to accumulate and store all sixteen 32-bit results of sixteen 16-bit multiplications. As the destination register still has a width of 256-bit, it can only hold eight 32-bit results in the default mode. Hence, `.even` and `.odd` flags are still needed for this mode. However, for `.lo`/`.hi` modes, we can now exploit the complete multiplier throughput.

A drawback of this solution is that the instruction behavior differs for 16-bit and 32-bit data types. This discrepancy might lead to software bugs if developers incorrectly assume uniform lane utilization across data types, resulting in silent logic errors where only a subset of lanes is updated or where stale values are inadvertently reused. Such bugs are particularly difficult to detect, as the instruction remains functionally correct at the instruction set architecture (ISA) level but produces incorrect results under specific data-type. Consequently, a real-world implementation should carefully weigh potential performance benefits against increased software-maintenance effort and correctness.

4.3 Hardware Implementation

In both variants described below, we replace the adders in BN-ALU and BN-MAC by buffer-bit adders introduced in Section 4.1.

OTBNV1: Basic Vector Multiply and Accumulate. For the hardware design of `bn.mulv` and `bn.mulv.l`, we replace the multiplier in BN-MAC by the multiplier described in Section 4.1. The additional overhead in the BN-MAC module of OTBNV1 over $\text{OTBN}_{\text{base}}$ includes large multiplexers to select the correct input words of source registers depending on data type, lane selection, scalar mode (with `.l`), and the original 64-bit multiplication path of the $\text{OTBN}_{\text{base}}$ BN-MAC functionality. In total, there are four decision bits in the input multiplexer tree.

Table 3: Resources for BN-MAC, BN-ALU, and OTBN (F_{max} in MHz, area in μm^2 $\times 10^3$).

Platform	FPGA Vivado					ASIC Genus		ASIC ORFS	
	LUT	DSP	CARRY4	FF	F_{max}	Area	F_{max}	Area	F_{max}
BN-MAC									
$OTBN_{base}$	2,166 ×0.49	16	143 × 0.90	312 ×0.61	58 ×1.87	61.9 × 0.73	890 ×2.07	250.8 × 0.77	67 × 1.76
$OTBN_{TW}$	4,413 ×1.00	16	159 × 1.00	508 ×1.00	31 ×1.00	84.8 × 1.00	430 ×1.00	326.4 × 1.00	38 × 1.00
OTBNV1	4,252 ×0.96	16	196 × 1.23	312 ×0.61	53 ×1.71	87.9 × 1.04	660 ×1.53	331.8 × 1.02	48 × 1.26
OTBNV2	5,771 ×1.31	16	264 × 1.66	624 ×1.23	53 ×1.71	104.9 × 1.24	640 ×1.49	413.2 × 1.27	48 × 1.26
BN-ALU									
$OTBN_{base}$	6,490 ×0.56	0	200 × 1.08	320 ×0.25	88 ×2.84	84.4 × 0.78	1,300 ×3.33	295.0 × 0.83	82 × 1.30
$OTBN_{KMAC}$	8,471 ×0.74	0	223 × 1.21	1,286 ×1.00	87 ×2.81	100.6 × 0.93	1,300 ×3.33	295.0 × 0.83	82 × 1.30
$OTBN_{TW}$	11,513 ×1.00	0	185 × 1.00	1,286 ×1.00	31 ×1.00	108.2 × 1.00	390 ×1.00	356.3 × 1.00	63 × 1.00
OTBNV1	12,095 ×1.05	0	231 × 1.25	1,286 ×1.00	77 ×2.48	120.0 × 1.11	1,300 ×3.33	414.6 × 1.16	86 × 1.37
OTBNV2	12,223 ×1.06	0	231 × 1.25	1,286 ×1.00	77 ×2.48	120.8 × 1.12	1,300 ×3.33	419.5 × 1.18	83 × 1.32
OTBN									
$OTBN_{base}$	32,971 ×0.80	16	770 × 1.00	15,652 ×0.93	37 ×1.85	426.3 × 0.81	680 ×2.06	1,910.4 × 0.92	46 × 1.64
$OTBN_{KMAC}$	35,174 ×0.85	16	793 × 1.03	16,635 ×0.99	38 ×1.90	443.4 × 0.84	660 ×2.00	1,910.4 × 0.92	46 × 1.64
$OTBN_{TW}$	41,146 ×1.00	16	771 × 1.00	16,830 ×1.00	20 ×1.00	525.5 × 1.00	330 ×1.00	2,075.6 × 1.00	28 × 1.00
OTBNV1	41,316 ×1.00	16	854 × 1.11	16,707 ×0.99	36 ×1.80	491.6 × 0.94	540 ×1.64	2,113.1 × 1.02	38 × 1.36
OTBNV2	42,976 ×1.04	16	922 × 1.20	17,036 ×1.01	35 ×1.75	507.3 × 0.97	530 ×1.61	2,203.7 × 1.06	38 × 1.36
[SOSK23]	38,170 ×0.93	49	—	16,603 ×0.99	—	—	—	—	—

Supporting the scalar word selection via the lane index for `bn.mulv.l` has a relatively small overhead since there is already $OTBN_{base}$ logic for the selecting a 64-bit word in the 256-bit input registers. We only need to add two levels to the binary selector tree, one to select the correct 32-bit words and one to select the correct 16-bit words.

Using existing data paths and resources of the $OTBN_{base}$ design, the double-length multiplication results are then directly fed to the accumulator adder. If the `.acc` flag is not set or the `.z` flag is set, the `ACC` register path is cleared; otherwise, the `ACC` register is added to the multiplication output and then updated with the accumulated result.

The preparation of the output data requires another large multiplexer, again, to preserve the original 64-bit multiplication data path, and to choose the correct data type, lane selection, and lane mode. In addition, the output-word select flags `.hi`/`.lo` are required for this multiplexer together with the first input operand. In total, there are five decision bits in the multiplexer tree.

OTBNV2: Second Accumulator Register. This instruction variant comes at little overhead compared to Version 1. The main additions are the second accumulator register and an additional data type for BN-MAC and BN-ALU input and output multiplexers.

Performance. Table 3 shows an overview of the resource cost of OTBNV1 for the BN-MAC and BN-ALU modules as well as the complete OTBN design. Although the two multiplexer trees for input and output handling introduce quite notable overhead, so does the state machine in the $OTBN_{TW}$ design. As a result, the overall resource cost of BN-MAC nearly breaks even. The maximum frequency of $OTBN_{TW}$ is significantly lower than those of other designs due to the poor performance of their sequential adder. Our buffer-bit vector adder needs some more CARRY4s but this can be mitigated using optimized adder designs discussed in Section 4.1.

The resource cost and performance impact of OTBNV2 is also shown in Table 3. The BN-MAC module shows a noticeable increase in adder and multiplexer logic as well as register usage compared to OTBNV1. However, F_{max} remains largely unaffected, as the additional `ACCH` register does not add latency to the longest path. The BN-ALU resource usage remains mostly unchanged since only minimal control logic is added to support software read and write access to the second accumulator. Overall, the impact on the complete OTBN module is moderate in both resource usage and maximum frequency.

```
1 bn.mulv.8S.even.acc.z.lo w2, w0, w1
2 bn.mulv.8S.even.lo       w2, w2, sw0.1
3 bn.mulv.8S.even.acc.hi   w2, w2, sw0.0
4 bn.mulv.8S.odd.acc.z.lo  w2, w2, w1
5 bn.mulv.8S.odd.lo        w2, w2, sw0.1
6 bn.mulv.8S.odd.acc.hi    w2, w2, sw0.0
7 bn.addvm.8S              w2, w2, bn0
```

(a) 32-bit inputs.

```
1 bn.mulv.16H.even.acc.z.lo w2, w0, w1
2 bn.mulv.16H.even.lo       w2, w2, sw0.1
3 bn.mulv.16H.even.acc.hi   w2, w2, sw0.0
4 bn.mulv.16H.odd.acc.z.lo  w2, w2, w1
5 bn.mulv.16H.odd.lo        w2, w2, sw0.1
6 bn.mulv.16H.odd.acc.hi    w2, w2, sw0.0
7 bn.addvm.16H              w2, w2, bn0
```

(b) 16-bit inputs.

Listing 1: Montgomery multiplications on OTBNV1 for 16-bit and 32-bit inputs.

We also used an additional industrial synthesis process with a TSMC 22nm PDK to confirm our results. For this process, OTBNV1 and OTBNV2 are 5% and 1% smaller compared to OTBN_{TW}, while $\text{OTBN}_{\text{base}}$ and $\text{OTBN}_{\text{KMAC}}$ are 9% and 6% smaller than OTBN_{TW}. These relative results align well with our numbers across different processes and tools, reinforcing that our designs are suitable for actual tapeout.

Comparison. The work in [SOSK23] extends OTBN with dedicated hardware accelerators for ML-{KEM,DSA}. They report FPGA resources for the complete OpenTitan SoC and for their individual OTBN accelerators. Their result shown in Table 3 is the sum of the base OTBN numbers with their accelerator resources so that it is comparable to our designs. They use fewer LUT and FF resources but significantly more DSPs, and do not report CARRY4 adder resources or frequency. They also report ASIC synthesis results, but only for the complete OpenTitan SoC with 22nm GlobalFoundries process. Therefore, we do not include those numbers in the comparison.

5 Software Integration

In this section, we discuss how the software implementation leverages the new multiplication instructions and evaluate their impact on the performance of the affected routines. For the remainder of this discussion, we set `w31=0` and refer to it as `bn0`. We also note that all software on OTBN in this work is written in assembly as in [AOP+25], since no dedicated toolchain for higher-level languages is available.

5.1 Modular Multiplication

OTBNV1: Basic Vector Multiply and Accumulate. Listing 1a shows how the new multiplication vector instruction can be used to perform eight Montgomery multiplications on OTBN for 32-bit inputs, while Listing 1b shows sixteen multiplications for 16-bit inputs.

Let `w0` and `w1` be vectors of n/d input elements. Let `sw0` contain the modulus q in its first d-bit slot `sw0.0` and $R = q^{-1} \bmod 2^d$ in the second d-bit slot `sw0.1`. Then n/d Montgomery multiplications are performed in seven cycles for d-bit inputs as follows:

1. Compute $c = ab$ and $[c]_d$ (Listing 1b, Line 1): `ACC` is emptied. The full $2d$-bit products of even-indexed elements of `w0` and `w1` are then put in `ACC`. Only their low d-bit parts are written to the even-indexed lanes of `w2`. Note that the untouched odd-indexed elements of `w0` are also written to `w2` (cf. Section 4.2).
2. Compute $[[c]_d R]_d$ (Listing 1b, Line 2): Even-indexed elements of `w2`—results of the first step—are multiplied with the constant R (`sw0.1`). This time, `ACC` is untouched. The low parts of the full products are written back to even-indexed lanes of `w2`. Again, the odd-indexed elements of `w2` are unchanged.
3. Compute $[c + [[c]_d R]_d q]^d$ (Listing 1b, Line 3): This time, `ACC` accumulates the full $2d$-bit products (`.acc`) of the even-indexed elements of `w2` and q (`sw0.0`). Then only the high d bits of `ACC`'s elements (`.hi`) are written to even-indexed elements of `w2`.

```
1 bn.mulv.8S.even.acc.z.lo w2, w0, w1
2 bn.mulv.8S.odd.acc.z.lo  w2, w2, w1
3 bn.mulv.8S.even.lo       w2, w2, sw0.1
4 bn.mulv.8S.odd.lo        w2, w2, sw0.1
5 bn.mulv.8S.even.acc.hi   w2, w2, sw0.0
6 bn.mulv.8S.odd.acc.hi    w2, w2, sw0.0
7 bn.addvm.8S              w2, w2, bn0
```

(a) 32-bit inputs.

```
1 bn.mulv.16H.acc.z.lo w0, w0, w1
2 bn.mulv.l.16H.lo     w0, w0, sw0.1
3 bn.mulv.l.16H.acc.hi w0, w0, sw0.0
4 bn.addvm.16H         w0, w0, bn0
```

(b) 16-bit inputs.

Listing 2: Montgomery multiplications on OTBNV2 for 16-bit and 32-bit inputs.

4. Step 1 to 3 are repeated for odd-indexed elements of the input operands using the `.odd` option. Note that the odd-indexed elements of `w0` have been copied to odd-indexed lanes of `w2` during Step 1 to 3 and that operating now on the odd-indexed elements keeps the already computed results in even positions intact.
5. Conditional subtraction (Listing 1b, Line 7): The results obtained in `w2` are now in $[0, 2q)$. Thus, q is conditionally subtracted from all words by using `bn.addvm` with 0.

OTBNV2: Second Accumulator Register. As described in Section 4.2, this instruction variant introduces an additional wide register, `ACCH`, and we conceptually treat the concatenation of `ACC` and `ACCH` as a single 512-bit accumulation register, where each element has a width of $2d$ bits. Since the available multiplier resources are exhausted for eight 32-bit multiplications, the instruction interface for the `.8S` variant remains unchanged from OTBNV1. However, the full products of even-/odd-indexed vector elements are now placed in the corresponding even/odd positions of this combined register. This applies to all 32-bit modes as well as the default 16-bit mode (i.e., without the `.lo`/`.hi` suffixes). For the 32-bit operations with `.acc` flag, the odd and even lanes are now entirely independent and therefore can be interleaved as shown in Listing 2a or they can be ordered as before in Listing 1a. For the 16-bit `.lo`/`.hi` modes, results of all sixteen multiplications fit within the 512-bit accumulator, enabling execution of sixteen Montgomery multiplications in just four cycles as shown in Listing 2b, approximately 42.86% faster than OTBNV1.

According to [AOP+25, Section 3.1.1, 6.2.2], n/d Plantard multiplications [Pla21] take 48 and 80 cycles for 16- and 32-bit inputs on $\text{OTBN}_{\text{KMAC}}$, while Montgomery multiplications need 12 cycles for both case on OTBN_{TW}. As we can see, with our new multiplication instructions, n/d Montgomery multiplications are about 41.7% faster for 32-bit case and up to 66.7% for 16-bit case, compared to OTBN_{TW}.

Performance. Table 4 presents the cycle counts in rows OTBNV1 and OTBNV2 for NTT, INTT and (pair-)pointwise multiplication in ML-KEM and ML-DSA. For ML-DSA, the multiplier consistently performs four 32-bit multiplications per cycle for both versions, yielding similar performance improvements of 27–28% over OTBN_{TW}. For ML-KEM, OTBNV1 achieves 28–30% speedup over OTBN_{TW} while OTBNV2 shows significant performance gain of 44–47% as Montgomery multiplication now takes only four cycles.

5.2 Lazy Reduction

Since modular multiplication is moved back to software, we can consider the lazy reduction technique in both NTT and INTT, i.e, delaying reduction of polynomial coefficients after a certain number of layers, given that the cost for reducing all coefficients after some layers is less than the total cost of reducing all for every layer. As we already have modular addition/subtraction with `bn.addvm`/`bn.subvm` for every (I)NTT layer, if the inputs of (I)NTT and results of Montgomery multiplications are both in $[0, q)$, the outputs of each layer will also be in that range. Hence, in our case, lazy reduction can only mean *removing the final conditional subtraction in Montgomery multiplication*. This idea also

Table 4: Cycle counts of NTT, INTT and (pair-)pointwise multiplication in ML-{KEM,DSA}. For two references, the first is for ML-KEM and the second for ML-DSA.

Platform	ML-KEM			ML-DSA		
	NTT	INTT	Pointw.	NTT	INTT	Pointw.
$\text{OTBN}_{\text{KMAC}}$ [AOP+25]	8,133 ×8.13	8,771 ×8.00	4,604 ×6.36	8,206 ×3.41	8,701 ×3.36	2,552 ×4.40
OTBN_{TW} [AOP+25]	1,000 ×1.00	1,096 ×1.00	724 ×1.00	2,404 ×1.00	2,587 ×1.00	580 ×1.00
OTBNV1	714 ×0.71	770 ×0.70	524 ×0.72	1,764 ×0.73	1,867 ×0.72	420 ×0.72
OTBNV1 with lazy reduction	657 ×0.66	722 ×0.66	484 ×0.67	1,635 ×0.68	1,755 ×0.68	388 ×0.67
OTBNV2	546 ×0.55	578 ×0.53	404 ×0.56	1,764 ×0.73	1,867 ×0.72	420 ×0.72
OTBNV2 with lazy reduction	489 ×0.49	530 ×0.48	364 ×0.50	1,635 ×0.68	1,755 ×0.68	388 ×0.67
OTBN [SOSK23]	1,454 ×1.45	1,726 ×1.57	1,448 ×2.00	1,972 ×0.82	2,244 ×0.87	768 ×1.32
OTBN [US25, Tur23]	4,356 ×4.36	—	—	10,763 ×4.48	13,943 ×5.39	9,714 ×16.75
Cortex-M4 [HZZ+22, HAZ+24]	4,474 ×4.47	4,684 ×4.27	2,422 ×3.35	8,066 ×3.36	8,388 ×3.24	1,931 ×3.33
RISC-V C908 [ZYHK25]	1,575 ×1.57	1,840 ×1.68	753 ×1.04	3,395 ×1.41	3,540 ×1.37	759 ×1.31
RISC-V [NDMZ+21]	18,488 ×18.49	18,488 ×16.87	—	18,554 ×7.72	21,375 ×8.26	—
RISC-V [FSS20]	1,935 ×1.94	1,930 ×1.76	—	—	—	—
RISC-V [LQYW24]	4,189 ×4.19	3,481 ×3.18	3,257 ×4.50	—	—	—

applies to (pair-)pointwise multiplication, which is used in matrix/vector-vector product. Unfortunately, this is not applicable due to the following reasons:

- In NTT, we compute subtraction in the form of $a - b\zeta$ for every layer, where a, $b \in [0, q)$ are its inputs and ζ is a root of unity. Without the final conditional subtraction, $b\zeta$ will be in $[0, 2q)$, leading $a - b\zeta$ to be in $(-2q, q)$, thus $(-q, q)$ after `bn.subvm` with `MOD` $= q$. This possibly negative result in $(-q, 0)$ again needs to be multiplied in the next layer—but the hardware multiplier only supports unsigned multiplication. The same problem happens from the second layer of INTT under similar analysis, given its inputs are also in $[0, q)$.
- In both ML-DSA and ML-KEM, (pair-)pointwise multiplications produce intermediate results that must ultimately be reduced to integers in $[0, q)$ before being passed to other functions. Skipping conditional subtractions during these multiplications can save instructions, but the benefit is offset by the need for a final reduction step.

We see that the main problem is this negative output in $(-q, 0)$. Resolving this means making the result going from $(-2q, 0)$ to ≥ 0 with `bn.subvm`. Our idea is to switch the modulus of modular addition/subtraction `MOD` to $2q$ before entering NTT and switch it back to q after INTT. Note that this does not affect the Montgomery multiplication, because the new multiplication instruction uses `sw0`/`sw1` to store the modulus q. The data flow from NTT to INTT is explained as below:

- We set `MOD` to $2q$.
- In the NTT, with a, $b \in [0, q)$ in the first layer, and using Montgomery multiplication without final conditional subtraction, $a+b\zeta$ (with `bn.addm`) and $a-b\zeta$ (with `bn.subm`) are in $[0, 2q)$. In subsequent layers, within each butterfly, Montgomery multiplication accepts inputs in this range since their product does not exceed $2d$ bits. As it reduces modulo q but omits the final conditional subtraction, its output lies in $[0, 2q)$. Thus, the results $a - b\zeta$ and $a + b\zeta$ of the butterfly are both in $[0, 2q)$ again.
- For ML-DSA's pointwise multiplication without final conditional subtractions, results are in $[0, 2q)$. The sum of inputs in this range with `bn.addvm` is again in $[0, 2q)$. Similarly for ML-KEM's pair-pointwise multiplication, as a_0b_0, $a_1b_1\zeta$, a_0b_1 and a_1b_0 are all in $[0, 2q)$, $a_0b_0 + a_1b_1\zeta$ and $a_0b_1 + a_1b_0$ with `bn.addvm` are again in $[0, 2q)$.
- Since most of the outputs of (pair-)pointwise products are inputs to INTT directly, we assume that INTT's inputs are in $[0, 2q)$. By applying the same reasoning for NTT, INTT's outputs are also in $[0, 2q)$. Because they are inputs to other functions

that require integers in $[0, q)$, we do not completely remove all final conditional subtractions, but keep some at the end of INTT. However, reducing the results to $[0, q)$ with `bn.addvm` means that `MOD` must be q again. Thus, we proceed to switch `MOD` back to q before these conditional subtractions and switch it again to $2q$ once done, for the next INTT layer.

- After INTT, we reset `MOD` to q.

The overhead of switching `MOD` is negligible compared to the gain of removing all the conditional subtractions for Montgomery multiplications. In particular, we save:

- $8 \times 7 = 56$ (ML-KEM) and $16 \times 8 = 128$ (ML-DSA) cycles/polynomial in NTT.
- $8 \times 6 - 3 = 45$ (ML-KEM) and $16 \times 7 - 6 = 106$ (ML-DSA) cycles/polynomial, where switching the modulus takes 3 and 6 cycles respectively, in INTT.
- 40 (ML-KEM) and 32 (ML-DSA) cycles/polynomial in (pair-)pointwise product.

The NTT and (pair-)pointwise multiplication can output values in $[0, 2q)$ as long as their results are immediately used in matrix/vector-vector product or the INTT afterwards. The exception is ML-KEM's key generation, where NTT outputs are passed to key packing routines, which require values in $[0, q)$. To reduce the outputs, we need sixteen `bn.addvm` instructions per polynomial. Unlike INTT, we omit these reductions from NTT, and place them in the key packing routine to avoid memory overhead, resulting in $2 \times K \times 16 \in \{64, 96, 128\}$ cycles more for key generation in total. Given that NTT saves 224 instructions already in ML-KEM-512, this still results in a net improvement.

Performance. Table 4 also presents the cycle counts for NTT, INTT and (pair-)pointwise with lazy reduction. Both versions show a further improvement of 4–5% from that of their non-lazy-reduced counterparts in Section 5.1 across ML-{KEM,DSA}, resulting in a total speedup of 32–34% for OTBNV1 and up to 52% for ML-KEM on OTBNV2 compared to OTBN_{TW}, demonstrating that our new multiplication instruction fully leverages the optimization potential for ML-KEM.

Compared to [SOSK23], our implementations of NTT and INTT for ML-DSA achieve performance improvements of 12% and 28% respectively, even though [SOSK23] proposed custom instruction set extensions specifically tailored for (I)NTT. We also significantly outperform [US25] and [Tur23]: the former underutilizes the full potential of big number arithmetic, while the latter employs the Kronecker+ technique from [BRvV22] which proves inefficient for OTBN. To provide further context, we also report performance of other architectures, including RISC-V and Cortex-M4. Among RISC-V implementations, the work in [ZYHK25] on the dual-issue C908 core with RVV[3] extension is the most comparable, as RVV also supports general vectorized multiplication. However, since [ZYHK25] performs Montgomery multiplication in software using four instructions, each with a latency of 4–5 cycles, and operates on only 128-bit vector registers, OTBN_{TW} is already faster, making our implementations even more efficient overall. Other ISEs proposed in [NDMZ+21, FSS20, LQYW24] are likewise tailored for these operations, as in [SOSK23], but are designed for 32-bit RISC-V cores, which explains their relatively lower performance.

Design Space Exploration. We also considered a variant "OTBNV3" that performs the final conditional subtraction of Montgomery multiplication (Line 4 of Algorithm 2.1) in hardware instead of software. The first option is to add another flag to `bn.mulv` that conditionally subtracts q from the top bits of the multiplication result. Using this mode for the last multiplication (Line 3 of Algorithm 2.1) then produces a fully reduced result in $[0, q)$. However, this requires an additional adder circuit in the critical path after the accumulator adder in BN-MAC, which affects both area and F_{max}. Since most field operations are performed in NTT or INTT, another option is to add an additional flag to `bn.addvm` and `bn.subvm` that first conditionally subtracts q from the input operand that holds the

[3] https://rvv-isadoc.readthedocs.io/en/latest/index.html

```
bn.mulv.l.8S a, a, c_s, 0
bn.mulv.l.8S b, b, c_s, 0
bn.trn2.16H  a, a, b
bn.shv.16H   r, a >> (16 - c_s)
```

(a) OTBN_{TW}.

```
bn.mulv.l.8S.even.hi a, a, c_m
bn.mulv.l.8S.odd.hi  a, a, c_m
bn.trn1.16H          r, r, a
```

(b) OTBNV1/OTBNV2.

Listing 3: Core operations of `poly_compress` on OTBN_{TW} and OTBNV1/OTBNV2.

multiplication result and then proceeds with the actual modular addition/subtraction. This also requires an additional adder circuit, in BN-ALU, which is not in the overall critical path and hence does not affect F_{max} of OTBN. However, Montgomery multiplications that are not followed by an addition or subtraction still require an explicit instruction to fully reduce the multiplication result.

We implemented both options in hardware and software and observed that ML-{KEM,DSA} achieve about the same cycle count on OTBNV3 as on OTBNV2 with lazy reduction but with higher resource cost for both options and lower F_{max} for the first option. Thus, we decided not to discuss these variants in the rest of this paper.

5.3 Ciphertext Packing in ML-KEM

Let c, c_s, c_a and c_m be constants. The core arithmetic of compressing and decompressing process in ML-KEM can be summed up in Equation (1) and Equation (2) respectively:

$$r = (((a \ll c_s) + c_a) \times c_m) \gg (c - c_s) \tag{1}$$

$$r = ((a \times q) + c_a) \gg c_s \tag{2}$$

For the compression of a polynomial, the constants (c, c_s, c_a, c_m) are $(32, 4, 1665, 80\,635)$ for ML-KEM-512/ML-KEM-768 and $(32, 5, 1664, 40\,318)$ for ML-KEM-1024. For the compression of a vector of polynomials, the constants (c, c_s, c_a, c_m) are $(42, 10, 1665, 1\,290\,167)$ for ML-KEM-512/ML-KEM-768 and $(42, 11, 1664, 645\,084)$ for ML-KEM-1024.

For the decompression of a polynomial, the constants (c_a, c_s) are $(8, 4)$ for ML-KEM-512/ML-KEM-768 and $(16, 5)$ for ML-KEM-1024. For the decompression of a vector of polynomials, the constants (c_a, c_s) are $(512, 10)$ for ML-KEM-512/ML-KEM-768 and $(1024, 11)$ for ML-KEM-1024.

Compression. Since c_m has a width of at least 16 bits in the compression of a polynomial, vectorized multiplication of 16-bit inputs with c_m are done as 32-bit multiplication (`.8S`) on OTBN_{TW}, as shown in Listing 3a, which takes four cycles for eight coefficients [AOP+25, Section 6.2.2]. As $c = 32$, the multiplication results are shifted 16 bits to the right by interleaving the high 16-bit parts of two result vectors using one `bn.trn2`, and arranging the outputs in correct order at the same time. Then the results are shifted right again by $16 - c_s$ bits to finish compression. In contrast, Listing 3b completes the combination of multiplication of a and $c_m \ll c_s$ together with the right shift in only two instructions for all eight coefficients. This combination is possible since our multiplication instruction can return either high or low parts of the full products, which is not the case for OTBN_{TW}. As a result, OTBN_{TW} needs eight cycles in total for these operations, while OTBNV1/OTBNV2 takes only two cycles. The performance difference of compression of vector of polynomials `polyvec_compress` between OTBN_{TW} and OTBNV1 is even more profound—since OTBN_{TW} needs to fall back to the original OTBN `bn.mulqacc` instruction for multiplication with c_m which now has at least 20 bits—concretely eighteen cycles for OTBN_{TW} and only four cycles for OTBNV1, as shown in Listing 4.

Decompression. Similar to compression, multiplication with q in Equation (2) on OTBN_{TW} (cf. Listing 4a) uses 32-bit multiplications. On OTBNV1 in Listing 5b, we

```
bn.mulqacc.z  a.0, c_m.0, 0
bn.mulqacc    a.1, c_m.0, 64
bn.mulqacc    a.2, c_m.0, 128
bn.mulqacc.wo a, a.3, c_m.0, 192
/* b,c,d is computed as a */
bn.trn2.8S    a, a, b
bn.trn2.8S    c, c, d
```

(a) OTBN_{TW}.

```
bn.mulv.l.8S.even.hi a, a, c_m
bn.mulv.l.8S.odd.hi  a, a, c_m
bn.mulv.l.8S.even.hi b, b, c_m
bn.mulv.l.8S.odd.hi  b, b, c_m
```

(b) OTBNV1/OTBNV2.

Listing 4: Core operations of `polyvec_compress` on OTBN_{TW} and OTBNV1/OTBNV2.

```
bn.mulv.l.8S a, a, q
bn.addv.8S   a, a, c_a
bn.shv.8S    a, a >> c_s
/* b is computed as a */
bn.trn1.16H r, a, b
```

(a) OTBN_{TW}.

```
bn.shv.16H                a, a << (16-c_s)
bn.wsrw                   ACC, c_a
bn.mulv.l.16H.even.acc.hi a, a, c_m
bn.wsrw                   ACCH, c_a
bn.mulv.l.16H.odd.acc.hi  a, a, c_m
```

(b) OTBNV1.

```
bn.shv.16H            a, a << (16-c_s)
bn.wsrw               ACC, c_a
bn.wsrw               ACCH, c_a
bn.mulv.l.16H.acc.hi  a, a, c_m
```

(c) OTBNV2.

Listing 5: Core operations of `poly(vec)_decompress` on OTBN_{TW} and OTBNV1/V2.

write $c_a \ll (16 - c_s)$ to `ACC` and use accumulation mode so that the addition is done together with the multiplication, which is not possible for OTBN_{TW} since their multiplication instruction does not support accumulation. We also use a similar trick as for `poly_compress`, i.e., to shift the inputs by $16 - c_s$ so that the high 16-bits of the products are exactly the final results of decompressing. OTBNV2 resembles OTBNV1 but saves one more instruction because this variant supports all sixteen 16-bit multiplications when `.lo`/`.hi` mode is enabled (cf. Listing 5c). In the end, OTBN_{TW} takes thirteen cycles to complete decompression of sixteen bytes, while OTBNV1 and OTBNV2 takes only five and four cycles respectively.

Performance. Table 5 reports the cycle counts for core compression and decompression routines in ML-KEM. For compression, we observe consistent speedups: 28% for `poly_compress` and 63% for `polyvec_compress`, on both OTBNV1 and OTBNV2. These gains stem from the exclusive use of 32-bit multiplications in these functions. For decompression, the improvements are even more pronounced—74% for OTBNV1 and 78% for OTBNV2. Since these routines contribute up to 12% of encapsulation and 19% of decapsulation runtime in OTBN_{TW} implementations, the results highlight that our proposed multiplication instructions not only provide substantial performance benefits for Montgomery multiplication but also are applicable across a broader range of operations.

6 Evaluation

Since our software implementation with lazy reduction performs better than without lazy reduction in all metrics, we focus the performance discussion on this variant and all software performance measurements reported in this section are with lazy reduction.

Performance. For the full-scheme benchmarks of ML-KEM, Table 6 shows that key generation, encapsulation, and decapsulation on OTBNV1 are approximately 6%, 8–9%, and 9–11% faster than on OTBN_{TW}, respectively. This translates to improvements of

Table 5: Cycle counts for ciphertext packing/unpacking functions of ML-KEM.

Platform	Compress		Decompress	
	poly	polyvec	poly	polyvec
$OTBN_{KMAC}$ [AOP+25]	115 ×6.39	115 ×3.29	99 ×5.21	99 ×5.21
$OTBN_{TW}$ [AOP+25]	18 ×1.00	35 ×1.00	19 ×1.00	19 ×1.00
OTBNV1	13 ×0.72	13 ×0.37	5 ×0.26	5 ×0.26
OTBNV2	13 ×0.72	13 ×0.37	4 ×0.21	4 ×0.21

Table 6: ML-KEM and ML-DSA full-scheme kilo-cycle counts rounded to three significant figures. Median of 10 000 iterations. If two references are given, the first one is for ML-KEM and the second one is for ML-DSA.

Platform	ML-KEM			ML-DSA		
	KeyGen.	Encap.	Decap.	KeyGen.	Sign	Verify
	ML-KEM-512			**ML-DSA-44**		
$OTBN_{KMAC}$ [AOP+25]	87.2 ×2.34	119 ×2.71	165 ×3.06	272 ×1.80	1 120 ×2.70	320 ×2.01
$OTBN_{TW}$ [AOP+25]	37.2 ×1.00	44.0 ×1.00	53.8 ×1.00	151 ×1.00	415 ×1.00	160 ×1.00
OTBNV1	34.8 ×0.94	40.0 ×0.91	47.7 ×0.89	140 ×0.93	348 ×0.84	145 ×0.91
OTBNV2	33.6 ×0.90	38.4 ×0.87	45.3 ×0.84	140 ×0.93	347 ×0.84	145 ×0.91
OpenTitan [SOSK23][3,a]	—	—	—	—	—	998 ×6.25
Cortex-M4 [HZZ+22][3,b][HAZ+24][3]	369 ×9.92	373 ×8.47	409 ×7.60	1 360 ×9.01	3 490 ×8.41	1 350 ×8.46
RISC-V [NDMZ+21][3]	420 ×11.27	438 ×9.95	101 ×1.88	1 590 ×10.57	5 880×14.19	1 700×10.66
RISC-V [FSS20][2][KSFS24][3]	150 ×4.03	193 ×4.38	205 ×3.81	593 ×3.94	1 910 ×4.60	651 ×4.08
RISC-V [LQYW24, LTQ+24][3]	622 ×16.70	785 ×17.82	713 ×13.26	542 ×3.60	845 ×2.04	563 ×3.53
	ML-KEM-768			**ML-DSA-65**		
$OTBN_{KMAC}$ [AOP+25]	157 ×2.23	197 ×2.49	258 ×2.80	440 ×1.68	1 930 ×2.74	495 ×1.92
$OTBN_{TW}$ [AOP+25]	70.6 ×1.00	78.8 ×1.00	92.0 ×1.00	262 ×1.00	704 ×1.00	258 ×1.00
OTBNV1	66.2 ×0.94	72.3 ×0.92	82.6 ×0.90	248 ×0.94	588 ×0.83	236 ×0.91
OTBNV2	64.0 ×0.91	69.6 ×0.88	78.8 ×0.86	248 ×0.94	589 ×0.84	236 ×0.91
OpenTitan [SOSK23][3,a]	—	—	—	—	—	1 490 ×5.76
Cortex-M4 [HZZ+22][3,b][HAZ+24][3]	603 ×8.55	627 ×7.95	673 ×7.32	2 390 ×9.12	5 570 ×7.92	2 300 ×8.91
RISC-V C908 [ZYHK25][3]	165 ×2.34	197 ×2.50	207 ×2.25	654 ×2.49	2 140 ×3.04	646 ×2.50
RISC-V [NDMZ+21][3]	695 ×9.84	732 ×9.28	130 ×1.42	2 970 ×11.33	10 200×14.50	2 960×11.47
RISC-V [FSS20][2][KSFS24][3]	273 ×3.88	326 ×4.13	340 ×3.70	1 070 ×4.07	3 250 ×4.62	1 130 ×4.36
RISC-V [LQYW24, LTQ+24][a]	988 ×14.00	1 240 ×15.70	1 130 ×12.32	902 ×3.44	1 330 ×1.89	919 ×3.56
	ML-KEM-1024			**ML-DSA-87**		
$OTBN_{KMAC}$ [AOP+25]	246 ×2.14	294 ×2.36	371 ×2.62	694 ×1.68	2 360 ×2.54	773 ×1.82
$OTBN_{TW}$ [AOP+25]	115 ×1.00	125 ×1.00	141 ×1.00	413 ×1.00	926 ×1.00	425 ×1.00
OTBNV1	108 ×0.94	115 ×0.92	128 ×0.91	388 ×0.94	787 ×0.85	392 ×0.92
OTBNV2	105 ×0.91	111 ×0.89	123 ×0.87	388 ×0.94	783 ×0.84	392 ×0.92
OpenTitan [SOSK23][3,a]	—	—	—	—	—	2 220 ×5.23
Cortex-M4 [HZZ+22][3,b][HAZ+24][3]	960 ×8.34	981 ×7.88	1 040 ×7.36	4 070 ×9.85	7 730 ×8.35	4 000 ×9.41
RISC-V [NDMZ+21][3]	1 090 ×9.48	1 130 ×9.04	160 ×1.13	5 000 ×12.11	13 300×14.40	5 130×12.09
RISC-V [FSS20][2][KSFS24][3]	350 ×3.04	405 ×3.26	425 ×3.00	1 780 ×4.32	4 360 ×4.70	1 850 ×4.35
RISC-V [LQYW24, LTQ+24][a]	1 540 ×13.41	1 850 ×14.86	1 720 ×12.15	1 530 ×3.71	2 070 ×2.23	1 560 ×3.67

[3] Round 3 Kyber/Dilithium [2] Round 2 Kyber/Dilithium [a] Parts of the execution on Ibex Core
[b] Benchmark results from [AOP+25, Section 7.1.2, Table 5] for speed-optimized m4fspeed version, since [HZZ+22] only provides numbers for stack-optimized m4fstack.

Table 7: ML-KEM and ML-DSA code size. "Text" refers to the size of the executable machine instructions of a program, i.e., the `.text` section. "Const" denotes the size of all constants used by the program, declared in the `.data` section. "I/O" refers to the size of the program's inputs and outputs, which are also declared in the `.data` section. All numbers refer to bytes.

Platform	ML-KEM			ML-DSA		
	Text	Const	I/O	Text	Const	I/O
	ML-KEM-512			**ML-DSA-44**		
$OTBN_{KMAC}$ [AOP+25]	14,888 ×1.66	2,688	3,360	20,736 ×1.12	4,768	9,600
$OTBN_{TW}$ [AOP+25]	8,944 ×1.00	1,600	3,360	18,452 ×1.00	2,624	9,600
OTBNV1	12,172 ×1.36	1,600	3,360	21,328 ×1.16	2,624	9,600
OTBNV2	10,228 ×1.14	1,600	3,360	21,328 ×1.16	2,624	9,600
	ML-KEM-768			**ML-DSA-65**		
$OTBN_{KMAC}$ [AOP+25]	15,428 ×1.64	2,688	4,832	20,724 ×1.13	4,768	12,608
$OTBN_{TW}$ [AOP+25]	9,436 ×1.00	1,600	4,832	18,340 ×1.00	2,624	12,608
OTBNV1	12,664 ×1.34	1,600	4,832	21,208 ×1.16	2,624	12,608
OTBNV2	10,720 ×1.14	1,600	4,832	21,208 ×1.16	2,624	12,608
	ML-KEM-1024			**ML-DSA-87**		
$OTBN_{KMAC}$ [AOP+25]	17,200 ×1.54	2,688	6,464	21,540 ×1.12	4,768	15,424
$OTBN_{TW}$ [AOP+25]	11,172 ×1.00	1,600	6,464	19,304 ×1.00	2,624	15,424
OTBNV1	14,372 ×1.29	1,600	6,464	22,180 ×1.15	2,624	15,424
OTBNV2	12,432 ×1.11	1,600	6,464	22,180 ×1.15	2,624	15,424

57–60%, 62–67%, and 66-71% over $OTBN_{KMAC}$, thanks to Montgomery multiplication requiring only six cycles when using lazy reduction (as discussed in Section 5.2). When the multiplication latency is further reduced to three cycles, OTBNV2 achieves speedups of 9–10%, 11–13%, and 13–16% over $OTBN_{TW}$ for the same three operations, corresponding to an additional gain of around 3–5% from that of OTBNV1. For ML-DSA, key generation and verification on OTBNV1 show performance gains of 6–7% and 5–7% over $OTBN_{TW}$, corresponding to 45–49% and 50–55% improvements over $OTBN_{KMAC}$. Signing benefits even more significantly, achieving a 15–17% speedup compared to $OTBN_{TW}$ and a 67–70% improvement over $OTBN_{KMAC}$. Since both OTBNV1 and OTBNV2 use the same multiplication instructions for ML-DSA, their performance gains are identical. Due to similar reasons as explained in Section 5.2, our implementations outperform all related works in Table 6. However, the most meaningful comparison is with the other OTBN-related implementations, while the remaining numbers serve primarily as general context.

Table 7 shows code size of ML-{KEM,DSA} for all security levels. Going from one to six instructions for a Montgomery multiplication on OTBNV1, ML-KEM shows 29–36% overhead in text size, while ML-DSA shows 15–16%. While this overhead remains unchanged for ML-DSA on OTBNV2 as previously explained, it drops to only 11–14% for ML-KEM as Montgomery now takes only three instructions. Given runtime improvements of up to 11% for ML-KEM on OTBNV1 and 16% on OTBNV2, as well as 16% for ML-DSA signing, this overhead is reasonable since both still fit into memory.

Time-Area Product. As discussed in Section 3.2, we evaluate the time-area products of ML-{KEM,DSA} across the ASIC platforms introduced in this work. We treat memory blocks as black box and do not include memory in the area estimates. Therefore, differences in logic area and consequently in the time-area product are more accentuated. Overall, the time-area information exclusively serves to investigate if the increased area requirements

Table 8: Full-scheme time-area product in $\mu m^2 \times$ ns relative to OTBN_{TW}, min and max over all ML-{KEM,DSA} security levels and operations (KeyGen., Encap., Sign, etc.).

Platform	ORFS	ORFS same F	Genus	Genus same F
$\text{OTBN}_{\text{KMAC}}$ [AOP+25]	×0.94–1.71	×1.54–2.82	×0.71–1.29	×1.41–2.58
OTBN_{TW} [AOP+25]	×1.00	×1.00	×1.00	×1.00
OTBNV1	×0.63–0.71	×0.85–0.96	×0.48–0.54	×0.78–0.88
OTBNV2	×0.65–0.74	×0.89–1.00	×0.50–0.57	×0.81–0.91

pay off in runtime gains. We report minima and maxima of the time-area products for all ML-{KEM,DSA} security levels relative to OTBN_{TW} with the ASIC targets ORFS and Genus. We exclude the FPGA target since there is no clear definition of "area" for an FPGA given the different types of area resources such as LUT, FF, CARRY4, DSP, BRAM, and routing resources. The first scenario, "ORFS" and "Genus", uses the time-area product based on wall-clock time, where area and F_{max} are taken from Table 3, and cycle counts from Table 7. The second scenario, "ORFS same F" and "Genus same F", assumes that all designs operate at a same frequency lower than their individual F_{max}. In this case, only the cycle counts serve as the time factor.

Table 8 shows that in the worst case, OTBNV1 and OTBNV2 achieve time-area products that are very close and even match that of OTBN_{TW} reaching 85–96% and 89–100% respectively, for "ORFS same F". This indicates that the additional resources and corresponding performance gains result in a time-area efficiency similar to that of OTBN_{TW}. However, in the best case for "Genus", we observe significantly better results, with time-area product improvements of 48–54% for OTBNV1 and 50–57% for OTBNV2. One reason for this discrepancy is that ORFS provides little automatic optimization for generic addition and multiplication synthesis, which ameliorates the performance impact of the less-optimal adders of [AOP+25]. Although some performance gains diminish when all designs are assumed to run at the same frequency, they remain substantial enough to offset the higher resource usage and maintain a competitive time-area balance.

7 Conclusion

We have demonstrated that shifting modular multiplication back to software through a re-designed vector multiplication instruction, combined with careful optimizations of minimal hardware components, most notably the adder, can yield significant performance improvements for both ML-KEM and ML-DSA. These benefits are achieved with moderate overhead, while enhancing the flexibility and generality of the ISE. We believe that the ISE, initially proposed in [AOP+25] and improved in this paper, is worth considering for a real-world deployment of post-quantum cryptography on OTBN. Promising directions for future work would be optimizing masked ML-KEM and ML-DSA using these vector instructions, as well as evaluating their resilience against side-channel attacks.

Acknowledgments

This project was partially funded by Academia Sinica via the Grand Challenge Project AS-GCP-114-M01 and by the Taiwanese National Science and Technology Council via grant NSTC 113-2221-E-001-024-MY3.

We would like to thank Andrew "bunnie" Huang for providing synthesis numbers with the TSMC 22nm process and Amin Abdulrahman for his valuable feedback on the text and the artifacts.

References

[ABCG20] Erdem Alkim, Yusuf Alper Bilgin, Murat Cenk, and François Gérard. Cortex-M4 optimizations for {R,M}LWE schemes. *IACR TCHES*, 2020(3):336–357, 2020. URL: https://tches.iacr.org/index.php/TCHES/article/view/8593, doi:10.13154/tches.v2020.i3.336-357.

[ACF+19] Tutu Ajayi, Vidya A. Chhabria, Mateus Fogaça, Soheil Hashemi, Abdelrahman Hosny, Andrew B. Kahng, Minsoo Kim, Jeongsup Lee, Uday Mallappa, Marina Neseem, Geraldo Pradipta, Sherief Reda, Mehdi Saligane, Sachin S. Sapatnekar, Carl Sechen, Mohamed Shalan, William Swartz, Lutong Wang, Zhehong Wang, Mingyu Woo, and Bangqi Xu. Toward an open-source digital flow: First learnings from the OpenROAD project. In *Proceedings of the 56th Annual Design Automation Conference 2019*, DAC '19. Association for Computing Machinery, 2019. doi:10.1145/3316781.3326334.

[AES01] Advanced Encryption Standard (AES). National Institute of Standards and Technology, NIST FIPS PUB 197, U.S. Department of Commerce, November 2001.

[AHKS22] Amin Abdulrahman, Vincent Hwang, Matthias J. Kannwischer, and Amber Sprenkels. Faster Kyber and Dilithium on the Cortex-M4. In Giuseppe Ateniese and Daniele Venturi, editors, *ACNS 2022*, volume 13269 of *LNCS*, pages 853–871. Springer, Cham, June 2022. doi:10.1007/978-3-031-09234-3_42.

[AOP+25] Amin Abdulrahman, Felix Oberhansl, Hoang Nguyen Hien Pham, Jade Philipoom, Peter Schwabe, Tobias Stelzer, and Andreas Zankl. Towards ML-KEM & ML-DSA on OpenTitan. In Marina Blanton, William Enck, and Cristina Nita-Rotaru, editors, *2025 IEEE Symposium on Security and Privacy*, pages 1–19. IEEE Computer Society Press, May 2025. doi:10.1109/SP61157.2025.00220.

[Bar86] Paul Barrett. Implementing the Rivest Shamir and Adleman public key encryption algorithm on a standard digital signal processor. In *Conference on the Theory and Application of Cryptographic Techniques*, pages 311–323. Springer, 1986. doi:10.1007/3-540-47721-7_24.

[BHK+22] Hanno Becker, Vincent Hwang, Matthias J. Kannwischer, Bo-Yin Yang, and Shang-Yi Yang. Neon NTT: Faster Dilithium, Kyber, and Saber on Cortex-A72 and Apple M1. *IACR TCHES*, 2022(1):221–244, 2022. doi:10.46586/tches.v2022.i1.221-244.

[BK82] Richard P. Brent and Hsiang-Tsung Kung. A regular layout for parallel adders. *IEEE Trans. Computers*, 31(3):260–264, 1982. doi:10.1109/TC.1982.1675982.

[BKS19] Leon Botros, Matthias J. Kannwischer, and Peter Schwabe. Memory-efficient high-speed implementation of Kyber on Cortex-M4. In Johannes Buchmann, Abderrahmane Nitaj, and Tajjeeddine Rachidi, editors, *AFRICACRYPT 19*, volume 11627 of *LNCS*, pages 209–228. Springer, Cham, July 2019. doi:10.1007/978-3-030-23696-0_11.

[BNG21] Luke Beckwith, Duc Tri Nguyen, and Kris Gaj. High-performance hardware implementation of CRYSTALS-Dilithium. In *2021 International Conference on Field-Programmable Technology (ICFPT)*, pages 1–10, 2021. doi:10.1109/ICFPT52863.2021.9609917.

[BRS22] Joppe W. Bos, Joost Renes, and Amber Sprenkels. Dilithium for memory constrained devices. In Lejla Batina and Joan Daemen, editors, *AFRICACRYPT 22*, volume 2022 of *LNCS*, pages 217–235. Springer, Cham, July 2022. doi:10.1007/978-3-031-17433-9_10.

[BRvV22] Joppe W. Bos, Joost Renes, and Christine van Vredendaal. Post-quantum cryptography with contemporary co-processors: Beyond Kronecker, Schönhage-Strassen & Nussbaumer. In Kevin R. B. Butler and Kurt Thomas, editors, *USENIX Security 2022*, pages 3683–3697. USENIX Association, August 2022. URL: https://www.usenix.org/conference/usenixsecurity22/presentation/bos.

[Che08] Li Chen. Hsiao-code check matrices and recursively balanced matrices. *CoRR*, abs/0803.1217, 2008. doi:10.48550/arXiv.0803.1217.

[CT65] James W Cooley and John W Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of computation*, 19(90):297–301, 1965. doi:10.2307/2003354.

[DKL+18] Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Dilithium: A lattice-based digital signature scheme. *IACR TCHES*, 2018(1):238–268, 2018. URL: https://tches.iacr.org/index.php/TCHES/article/view/839, doi:10.13154/tches.v2018.i1.238-268.

[DT05] Albert Danysh and Dimitri Tan. Architecture and implementation of a vector/SIMD multiply-accumulate unit. *IEEE Trans. Computers*, 54(3):284–293, 2005. doi:10.1109/TC.2005.41.

[FSS20] Tim Fritzmann, Georg Sigl, and Johanna Sepúlveda. RISQ-V: Tightly coupled accelerators for post-quantum cryptography. *IACR TCHES*, 2020(4):239–280, 2020. URL: https://tches.iacr.org/index.php/TCHES/article/view/8683, doi:10.13154/tches.v2020.i4.239-280.

[GKS21] Denisa O. C. Greconici, Matthias J. Kannwischer, and Amber Sprenkels. Compact Dilithium implementations on Cortex-M3 and Cortex-M4. *IACR TCHES*, 2021(1):1–24, 2021. URL: https://tches.iacr.org/index.php/TCHES/article/view/8725, doi:10.46586/tches.v2021.i1.1-24.

[GS66] W Morven Gentleman and Gordon Sande. Fast Fourier transforms: for fun and profit. In *Proceedings of the November 7-10, 1966, fall joint computer conference*, pages 563–578, 1966. doi:10.1145/1464291.1464352.

[GSM17] Hannes Gross, David Schaffenrath, and Stefan Mangard. Higher-order side-channel protected implementations of Keccak. Cryptology ePrint Archive, Report 2017/395, 2017. URL: https://eprint.iacr.org/2017/395.

[Har03] David Harris. A taxonomy of parallel prefix networks. In *The Thrity-Seventh Asilomar Conference on Signals, Systems & Computers, 2003*, volume 2, pages 2213–2217, 2003. doi:10.1109/ACSSC.2003.1292373.

[HAZ+24] Junhao Huang, Alexandre Adomnicai, Jipeng Zhang, Wangchen Dai, Yao Liu, Ray C. C. Cheung, Çetin Kaya Koç, and Donglong Chen. Revisiting Keccak and Dilithium implementations on ARMv7-M. *IACR TCHES*, 2024(2):1–24, 2024. doi:10.46586/tches.v2024.i2.1-24.

[HZZ+22] Junhao Huang, Jipeng Zhang, Haosong Zhao, Zhe Liu, Ray C. C. Cheung, Çetin Kaya Koç, and Donglong Chen. Improved plantard arithmetic for lattice-based cryptography. *IACR TCHES*, 2022(4):614–636, 2022. doi:10.46586/tches.v2022.i4.614-636.

[Kob87] Neal Koblitz. Elliptic curve cryptosystems. *Mathematics of Computation*, 48(177):203–209, 1987. doi:10.1090/S0025-5718-1987-0866109-5.

[KSFS24] Patrick Karl, Jonas Schupp, Tim Fritzmann, and Georg Sigl. Post-quantum signatures on RISC-V with hardware acceleration. *ACM Transactions on Embedded Computing Systems*, 23(2):30:1–30:23, 2024. doi:10.1145/3579092.

[LDK+22] Vadim Lyubashevsky, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Peter Schwabe, Gregor Seiler, Damien Stehlé, and Shi Bai. CRYSTALS-DILITHIUM. Technical report, National Institute of Standards and Technology, 2022. available at https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022.

[LQYW24] Lu Li, Guofeng Qin, Yang Yu, and Weijia Wang. Compact instruction set extensions for Kyber. *IEEE Transactions on Computer-Aided Design ofIntegrated Circuits and Systems*, 43(3):756–760, 2024. doi:10.1109/TCAD.2023.3327104.

[LTQ+24] Lu Li, Qi Tian, Guofeng Qin, Shuaiyu Chen, and Weijia Wang. Compact instruction set extensions for Dilithium. *ACM Transactions on Embedded Computing Systems*, 23(2):23:1–23:21, 2024. doi:10.1145/3643826.

[Lyu09] Vadim Lyubashevsky. Fiat-Shamir with aborts: Applications to lattice and factoring-based signatures. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 598–616. Springer, Berlin, Heidelberg, December 2009. doi:10.1007/978-3-642-10366-7_35.

[Mil86] Victor S. Miller. Use of elliptic curves in cryptography. In Hugh C. Williams, editor, *CRYPTO'85*, volume 218 of *LNCS*, pages 417–426. Springer, Berlin, Heidelberg, August 1986. doi:10.1007/3-540-39799-X_31.

[Mon85] Peter L. Montgomery. Modular multiplication without trial division. *Mathematics of Computation*, 44(170):519–521, 1985. doi:10.1090/S0025-5718-1985-0777282-X.

[Nat24a] National Institute of Standards and Technology. FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard, 2024. doi:10.6028/NIST.FIPS.203.

[Nat24b] National Institute of Standards and Technology. FIPS 204: Module-Lattice-Based Digital Signature Standard, 2024. doi:10.6028/NIST.FIPS.204.

[NDD+25] Trong-Hung Nguyen, Duc-Thuan Dam, Phuc-Phan Duong, Binh Kieu-Do-Nguyen, Cong-Kha Pham, and Trong-Thuc Hoang. Efficient hardware implementation of the lightweight CRYSTALS-Kyber. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 72(2):610–622, 2025. doi:10.1109/TCSI.2024.3443238.

[NDMZ+21] Pietro Nannipieri, Stefano Di Matteo, Luca Zulberti, Francesco Albicocchi, Sergio Saponara, and Luca Fanucci. A RISC-V post quantum cryptography instruction set extension for number theoretic transform to speed-up CRYSTALS algorithms. *IEEE Access*, 9:150798–150808, 2021. doi:10.1109/ACCESS.2021.3126208.

[Ope23] OpenTitan Team. Datasheet — OpenTitan documentation, 2023. URL: https://opentitan.org/book/doc/introduction.html.

[Ope24] OpenTitan. Opentitan big number accelerator (OTBN) technical specification, 2024. URL: https://opentitan.org/book/hw/ip/otbn/index.html#security-features.

[Pla21] Thomas Plantard. Efficient word size modular arithmetic. *IEEE Transactions on Emerging Topics in Computing*, 9(3):1506–1518, July 2021. URL: https://thomas-plantard.github.io/pdf/Plantard21.pdf, doi:10.1109/TETC.2021.3073475.

[RSA78] Ronald L. Rivest, Adi Shamir, and Leonard M. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the Association for Computing Machinery*, 21(2):120–126, February 1978. doi:10.1145/359340.359342.

[SAB+22] Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, Damien Stehlé, and Jintai Ding. CRYSTALS-KYBER. Technical report, National Institute of Standards and Technology, 2022. available at https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022.

[SOSK23] Tobias Stelzer, Felix Oberhansl, Jonas Schupp, and Patrick Karl. Enabling Lattice-Based Post-Quantum Cryptography on the OpenTitan Platform. In *Proceedings of the 2023 Workshop on Attacks and Solutions in Hardware Security*, ASHES '23, pages 51–60. Association for Computing Machinery, 2023. doi:10.1145/3605769.3623993.

[Tur23] Horia Turcuman. Speeding-up Post-Quantum Cryptography on an RSA Co-Processor. Master's thesis, Technical University of Munich, September 2023. URL: https://github.com/horiaionut/kroneker-plus-on-otbn/blob/5576d7b035f5fe55a7199987ea05613d4aa913e7/paper.pdf.

[US25] Emma Urquhart and Frank Stajano. Acceleration of core post-quantum cryptography primitive on open-source silicon platform through hardware/-software co-design. In *Cryptology and Network Security*, pages 144–161. Springer Nature Singapore, 2025. URL: https://www.cl.cam.ac.uk/~fms27/papers/2024-UrquhartStajano-acceleration.pdf.

[Wal64] Christopher S. Wallace. A suggestion for a fast multiplier. *IEEE Trans. Electron. Comput.*, 13(1):14–17, 1964. doi:10.1109/PGEC.1964.263830.

[ZXXH22] Yifan Zhao, Ruiqi Xie, Guozhu Xin, and Jun Han. A high-performance domain-specific processor with matrix extension of RISC-V for Module-LWE applications. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 69(7):2871–2884, July 2022. doi:10.1109/TCSI.2022.3162593.

[ZYHK25] Jipeng Zhang, Yuxing Yan, Junhao Huang, and Çetin Kaya Koç. Optimized software implementation of Keccak, Kyber, and Dilithium on RV32,64IMBV. *IACR TCHES*, 2025(1):632–655, 2025. doi:10.46586/tches.v2025.i1.632-655.